

Searching for the Ideal Number of Threads on Asymmetric Multiprocessors

Marcelo K. Moori, Hiago Mayk G. de A. Rocha, Arthur F. Lorenzon and Antonio Carlos S. Beck

Institute of Informatics - Federal University of Rio Grande do Sul, Brazil

E-mail: {mkmoori, hmgarocha, afflorenz, caco}@inf.ufrgs.br

Abstract—Asymmetric multicore processors (AMP) combine high-performance cores with more energy-efficient ones, capitalizing on the diverse performance demands of modern devices (e.g., smartphones and tablets). Although hardware players have been designing more powerful AMPs for desktop and server computers, such as the Apple M1 and Intel Alder Lake family, these processors impose new challenges on parallel computing researchers on how to properly use them to their fullest. As we show in this paper, the best number of threads and which combination of the kind of cores will very likely change with the applications and user requirements, i.e., performance, energy, or energy-delay product (EDP). Based on that, we propose *HyTuning*, a heuristic algorithm that improves the above-mentioned requirements (as a user choice) by automatically adjusting the number of threads and selecting the most appropriate cores to execute a given application on an Intel Core i9-12900K Alder Lake processor. Our experimental results considering eleven parallel benchmarks show that *HyTuning* outperforms the other three compared strategies on average by $1.40\times$, $2.08\times$, and $2.05\times$ optimizing for execution time, energy, and EDP, respectively.

Index Terms—Asymmetric Multicore, Hybrid processors, Number of Threads, Parallel Applications, Intel Alder Lake

I. INTRODUCTION

Since energy efficiency became one of the most critical constraints of processor design, asymmetric multicore processors (AMP), like the ARM's big.LITTLE, emerged as the mainstream technology in most current portable devices (e.g., smartphones and tablets). They comprise general purposes high-performance (*big*) and energy-efficient (*small*) cores. These cores are different in their microarchitectures while sharing a single instruction-set architecture (ISA), making them capable of meeting the performance and energy requirements of modern and legacy mobile applications [1]. AMP's main benefit is to extend the battery life of portable devices and keep their power dissipation low, with the big cores executing foreground and latency-sensitive tasks. In contrast, the power-efficient small cores run the background and low-priority tasks. Inspired by that, prominent hardware designers have moved AMPs beyond the mobile niche towards the desktop/server segments, like the Apple M1 SoC and, more recently, the Intel Alder Lake processor [2]. Intel's processor is more suitable for high-performance workloads and has been proving itself to be effective for applications like graphics-intensive games and scientific simulation software [1].

Despite the benefits of AMPs in meeting the performance and energy demands of different applications, a limitation

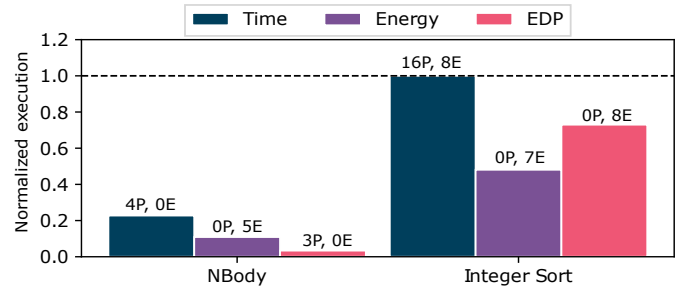


Fig. 1. Improvement of the best configuration of NBody and Integer Sort algorithms in regard to execution time (Time in Blue), Energy (in purple) and EDP (in pink). Results are normalized by the respective metrics obtained by the regular parallel execution (our baseline), which uses the maximum number of threads. Thus, the lower, the better.

remains w.r.t. the parallel applications' scalability, precluding the extraction of their full potential on such architectures. Many parallel applications are load unbalanced by nature, but the existence of different (big and small) cores potentiates that. Therefore, many times, executing a given parallel application with the maximum number of threads (the default behavior) may result in performance degradation or higher energy consumption. Therefore, choosing the proper number of threads of big and small cores to execute a given parallel application poses a significant challenge, considering the increased complexity of dealing with AMP systems [2]. However, in AMPs systems, with their intrinsic heterogeneous nature, the number of active threads matters as to which cores will be used to execute them.

Although improving the parallel applications' performance by adjusting the number of threads has been widely applied in the literature [3]–[10], *none of these works consider AMPs*. Therefore, we perform a design space exploration (DSE) to analyze the optimization potentials of tuning the number of threads of several parallel applications on a modern AMP. We also show that the best number of threads for a given application differs and strongly depends on whether one intends to reduce the execution time, energy consumption, or the trade-off between them, represented by the energy-delay product (EDP) [11].

To illustrate the findings mentioned above, Fig. 1 shows the improvement achieved by executing the *NBody* and *Integer Sort* applications (each set of bars) with their best number of threads for execution time, energy, and EDP (in different bars). Experiments were run on an Intel Core i9-12900K Alder Lake processor, which can execute up to 16 threads on big and 8 threads on small cores (referred to as P-core and E-

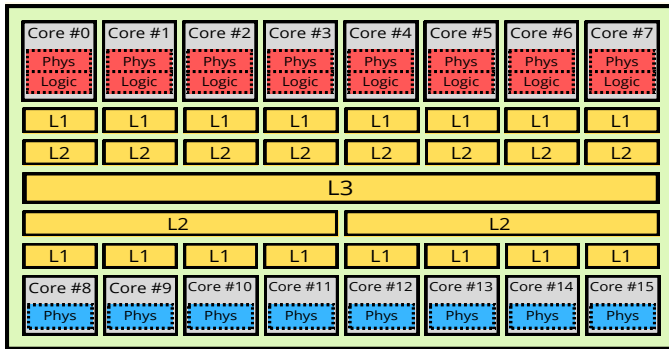


Fig. 2. Simplified view an AMP processor: Intel Core i9-12900K Alder Lake. The red-filled spaces represent P-Cores while the blue-filled are the E-cores.

cores by Intel, respectively) – see more details in section II. Results are normalized by the regular parallel execution using the maximum number of threads, so the lower, the better. For *NBody*, while the best number of threads for execution time is to use four performance cores and no energy ones (4P, 0E) when optimizing for energy and EDP the best configuration is (0P, 5E) and (3P, 0E). Such solutions deliver improvements of 77.24%, 89.11%, and 96.62% in the respective metrics. The very same behavior happens for *Integer Sorting*.

Considering such significant optimization potential and leveraging the knowledge obtained from the DSE, we propose **HyTuning**, a heuristic algorithm for optimizing the application performance requirements (execution time, energy, or EDP) by adjusting the number of threads on P-cores and E-cores of Alder Lake processors. In a nutshell, the main contributions of this paper are:

- A DSE considering eleven parallel applications executed on a modern Intel Alder Lake processor, raising significant findings of the behaviors of the applications' execution on such machine;
- **HyTuning**, a heuristic algorithm to find the ideal number of threads in P-cores and E-cores to execute a given parallel application in the target processor.

Our results evaluating **HyTuning** in comparison to three state-of-the-art strategies showed that our proposal improves by up to $1.79\times$, $2.66\times$, and $2.46\times$, the execution time, energy, and EDP in the average of the evaluated applications.

II. BACKGROUND AND RELATED WORK

Limitations of parallel scalability. As motivated in Fig.1 (see section I), reducing the number of threads provides a better use of the AMP's resource utilization, bringing benefits in execution time, energy, and EDP. They are related to parallel scalability limitations suffered by most of the applications due to software and hardware issues [6], [7], [9], such as:

- **Saturation of functional units.** Although SMT can benefit applications with low instruction level parallelism (ILP), it may result in extra idle cycles if an individual thread presents enough ILP to issue instructions in most of the core's functional units;
- **Off-chip bus saturation.** The off-chip bus bandwidth is limited by the number of I/O pins, which does not increase as the number of cores increases. Thus, when it saturates, there are no further improvements by increasing the number of threads;

- **Overhead of data synchronization.** The overhead of threads' communication through shared regions on L3 and main memories may also become a bottleneck;

- **Number of accesses of shared memory.** As a single thread must execute a critical section (region of code) at once, when the number of threads increases, more threads must be serialized inside the critical sections, increasing the synchronization time.

Adjusting number of threads. Since optimizing the number of threads is not straightforward, several works propose and evaluate different strategies on homogeneous architectures (i.e., not Asymmetric Processors). Lee et al. [12] propose Thread Tailor, a dynamic compiler that weaves threads together to reduce the total number of threads, optimizing system efficiency. The work of Wang et al. [13] presents NuCore, a model which predicts the optimal core allocations and number of threads for Non-Uniform Memory Access (NUMA) machines. De Sensi et al. [4] implement an algorithm that, given an optimization metric, gives the ideal number of threads and configuration without the need for prior execution. Lastly, Malave et al. [14] propose a method that predicts the best number of threads, optimizing a fitness value determined by operating system factors such as context switching rate.

Asymmetric Multicore Processors (AMP). As just explained in section I, the design of AMPs couple micro-architecturally different general purpose big high-performance cores and small energy-efficient ones. As an example of an AMP, we show, in Fig. 2, a simplified view of the Intel Core i9-12900K Alder Lake processor (used in our experiments). As mentioned, Intel refers to high-performance cores as P-cores and energy efficiency cores as E-cores. This processor comprises 16 cores: 8 P-cores (cores 0-7) and 8 E-cores (cores 8-15), where the P-cores support simultaneous multithreading (SMT) execution so that each P-core can execute two threads simultaneously each. Given that, the whole processor supports 24 threads. While each P-core has a private 1.25 MB L2 cache, the E-cores are grouped in two 4-core clusters (8-11 and 12-15), each sharing a 2 MB L2 cache. All cores have private L1 caches for instruction (L1i) and data (L1d) and share a 30MB L3.

The state-of-the-art of AMP comprises works that focus on the design of new micro-architectures [15] and on the scheduling of workloads to different types of cores [16]. However, only a few works explore the execution of parallel applications on Alder Lake processors. The work of Yue et al. [17] proposes an IPC-based scheduler to dispatch workloads. Bilbao et al. [2] propose a framework that enables and facilitates the development of custom scheduling-related support in the Linux kernel. Saez et al. [1] compare the Thread Director (i.e., a scheduler designed specifically for Alder Lake processors) to a set of hardware-counter-based asymmetric schedulers.

Our contributions. While works in the AMP literature have been proposing improvements in AMP's microarchitecture and schedules, to the best of our knowledge, we are the first to analyze the optimization potential of adjusting the number of threads of parallel applications executing on AMP. In this paper, we fully exploit the problem search space, delivering solutions that outperform the regular ways of executing ap-

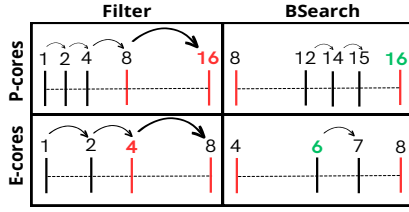


Fig. 3. *HyTuning*'s execution flow when optimizing BT for performance.

plications on the representative AMP processor, Intel Alder Lake.

III. HYTUNING HEURISTIC

HyTuning is a heuristic method for finding the best number of threads to run any parallel application on the P-cores and E-cores of AMP Alder Lake processors, considering a specific optimization metric, e.g., execution time, energy consumption, or EDP. We give an overview of *HyTuning* in Fig. 3 when it optimizes the *Block Tri-diagonal* (BT) application for performance. In short, given the application binary and the optimization metric, *HyTuning* iteratively learns by performing the following procedures:

- *Filter*, which increases the number of threads by a power of 2 until there is no improvement, cutting significantly the search space. This procedure returns a promising region, i.e., where the best number of threads will be (region delimited by the bars in red);
- *Binary Search*, which receives the promising region and finds the best number of threads (numbers in green), by logarithmically cutting the search space.

We designed *HyTuning* based on our findings on the DSE that will be discussed in Section V-A, so the *Filter* and *Binary Search* procedures are respectively performed two times (or three times for optimizing EDP) on different orders (i.e., first searching on P-core and after on E-cores, as shown in Fig.3, and vice-versa), depending on the optimization metric. In the following, we detail the *HyTuning*'s main pseudocode (Algorithm 1), as well as the *Filter* and *Binary Search* pseudocodes (Algorithms 3 and 4). For simplicity, we summarize, in Algorithm 1, a sequence of *Filter* and *Binary Search* as a *Search* procedure detailed in Algorithm 2.

HyTuning's main procedure. As shown in Algorithm 1, *HyTuning* receives as **inputs** the path to the application's binary to be optimized (*app*) along with the description of the objective metric (*opt*). First, it initializes the variables that stores the current solution ((p, e)) and the best values v_{best} (lines 1 and 2) and gets the inferior (inf_p and inf_e) and superior (sup_p and sup_e) limits for the *Search* (S) procedure (lines 3 and 4), which are the indexes of each core in the target multicore processor. The procedures *GetPcoresIndexes*() and *GetEcoresIndexes*() return the lowest and highest indexes of the P-cores and E-cores, respectively. After that, *HyTuning* applies the S procedure either on P-cores or E-cores, depending on the optimization metric *opt* (lines 5-19). Notice that the '_' symbol indicates when the S's second returned value is unimportant in the optimization. In the case of $opt == \text{"Performance"}$ (lines 5-7), it searches the best number of cores first in P-cores, then in E-cores. When $opt == \text{"Energy"}$ (lines 8-10), it searches first in E-cores,

Algorithm 1 *HyTuning*

Input: *app* $\leftarrow$ Path to the application's binary;
opt $\leftarrow$ Optimization metric ["Performance", "Energy", or "EDP"].

Output: (p, e) , the best tuple of number of threads.

```

1:  $(p, e), (p', e') \leftarrow (0, 0)$ 
2:  $v_{best} \leftarrow \text{infinity}$ 
3:  $inf_p, sup_p \leftarrow \text{GetPcoresIndexes}()$ 
4:  $inf_e, sup_e \leftarrow \text{GetEcoresIndexes}()$ 
5: if ( $opt == \text{"Performance"}$ ) then ▷ Optimize for exec. time
6:    $(p, e), v_{best} \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'P'}, inf_p, sup_p, v_{best})$ 
7:    $(p, e), \_ \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'E'}, inf_e, sup_e, v_{best})$ 
8: else if ( $opt == \text{"Energy"}$ ) then ▷ Optimize for energy
9:    $(p, e), v_{best} \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'E'}, inf_e, sup_e, v_{best})$ 
10:   $(p, e), \_ \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'P'}, inf_p, sup_p, v_{best})$ 
11: else ▷ Optimize for EDP
12:    $(p, e), edp_p \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'P'}, inf_p, sup_p, v_{best})$ 
13:    $(p', e'), edp_e \leftarrow S(\text{app}, \text{opt}, (p', e'), \text{'E'}, inf_e, sup_e, v_{best})$ 
14:   if ( $edp_p < edp_e$ ) then
15:      $(p, e), \_ \leftarrow S(\text{app}, \text{opt}, (p, e), \text{'E'}, inf_e, sup_e, edp_p)$ 
16:   else
17:      $(p, e), \_ \leftarrow S(\text{app}, \text{opt}, (p', e'), \text{'P'}, inf_p, sup_p, edp_e)$ 
18:   end if
19: end if
20: return  $(p, e)$ 
```

Algorithm 2 *Search* (S)

Input: *app* $\leftarrow$ Path to the application's binary;
opt $\leftarrow$ Optimization metric ["Performance", "Energy", or "EDP"];
 $(p, e) \leftarrow$ Solution's tuple;
cores $\leftarrow$ Cores' type ['P', 'E'];
inf $\leftarrow$ Inferior limit;
sup $\leftarrow$ Superior limit;
 $v_{best} \leftarrow$ best value for *opt* metric found so far.

Output: $(p, e), v_{best}$, being the current best number of threads and its value for *opt* metric, respectively.

```

1:  $inf, sup, (p, e), v_{best} \leftarrow F(\text{app}, \text{opt}, (p, e), \text{cores}, sup, v_{best})$ 
2:  $(p, e), v_{best} \leftarrow BS(\text{app}, \text{opt}, (p, e), \text{cores}, inf, sup, v_{best})$ 
3: return  $(p, e), v_{best}$ 
```

to then in P-cores. Otherwise, it will optimize for EDP (*opt* equals "EDP") in lines 11 to 19. As *HyTuning* does not know in advance which kind of core brings more benefits in the EDP metric of a given application, it tunes the number of threads in both kinds on cores and compares the resulting EDP values (edp_p and edp_e) to know which is better. If it is better to optimize on P-cores first, it optimizes on E-cores next (line 15). On the contrary, if it is better to optimize on E-core first, it optimizes on P-cores next (line 17). Finally, as the **output** (line 20), *HyTuning* returns the best number of threads to execute the given application on the target system (p, e) .

***Search* (S).** It simply encapsulates the *Filter* (F) and *Binary Search* (BS) procedures, in this exact order, as detailed on Algorithm 2.

***Filter* (F).** This procedure is described in Algorithm 3. In addition to the path to the application being optimized *app* and the description of the metric for optimization *opt*, it receives as **inputs** the current solution of the number of threads in P-cores and E-cores (p, e) , the kind of cores to consider currently *cores*, the superior *sup* limit of the current kind (E or P) of cores, and the best value of the respective metric found so far v_{best} . The *Filter*, firstly, initializes the control variables to guide the search (lines 1-4) and iteratively (lines 5-21) increases the number of threads in (p, e) configuration until it gets to the superior limit, or until the current metric does not

Algorithm 3 *Filter (F)*

Input: $app \leftarrow$ Path to the application's binary;
 $opt \leftarrow$ Optimization metric ["Performance", "Energy", or "EDP"];
 $(p, e) \leftarrow$ Solution's tuple;
 $cores \leftarrow$ Cores' type ['P', 'E'];
 $sup \leftarrow$ Superior limit;
 $v_{best} \leftarrow$ best value for opt metric found so far.

Output: $t_{prev}, t_{curr}, (p', e'), v_{best}$, being the inferior limit, the superior limit, the current best number of threads and its value for opt metric, respectively.

```
1:  $t_{prev}, t_{curr} \leftarrow 1$ 
2:  $v_{prev}, v_{curr} \leftarrow infinity$ 
3:  $exp \leftarrow 0$ 
4:  $(p', e') \leftarrow (p, e)$ 
5: while ( $t_{curr} < sup$  and  $v_{prev} \geq v_{curr}$ ) do
6:    $t_{prev} \leftarrow t_{curr}$ 
7:    $t_{curr} \leftarrow 2^{exp}$ 
8:   if ( $cores == 'P'$ ) then                                ▷ P-cores
9:      $p \leftarrow t_{curr}$ 
10:  else                                                    ▷ E-cores
11:     $e \leftarrow t_{curr}$ 
12:  end if
13:   $v \leftarrow RunProgram(app, opt, (p, e))$ 
14:   $v_{prev} \leftarrow v_{curr}$ 
15:   $v_{curr} \leftarrow v$ 
16:  if ( $v < v_{best}$ ) then
17:     $v_{best} \leftarrow v_{curr}$ 
18:     $(p', e') \leftarrow (p, e)$ 
19:  end if
20:   $exp \leftarrow exp + 1$ 
21: end while
22: return  $t_{prev}, t_{curr}, (p', e'), v_{best}$ 
```

show improvement over the previously computed (condition $t_{curr} < sup$ and $v_{prev} \geq v_{curr}$ in line 5). The (p, e) configuration, in turn, gets updated in each loop by setting the number of threads according to the kind on cores $cores$ being currently searched, following the sequence of powers of 2. The procedure ends by returning the inferior and superior limit of the filtered region t_{prev} and t_{curr} and the best configuration and metrics yet found (p', e') and v_{best} .

Binary Search (BS). This procedure works as presented in Algorithm 4. It receives the same inputs as *Filter* procedure, but updated by such procedure, in addition to the found inferior limit inf . Firstly, it initializes (lines 1-2) the (p', e') to store the best solutions found during the search and the control variable $invert$. Because the search space always gets cut in the direction of the best value, the $invert$ variable is used to correctly cut the search space in the case *Filter* returns sup as the best number of threads. This procedure, then (lines 3-17), tests if the inferior and superior limits are equal ($inf == sup$), and if true, the algorithm returns the sup as the best number for p or e along with v_{best} ; right after, it tests if the corresponding core in $cores$ is equal to sup , and if true, it inverts the logic on updating the sup and inf variables on the search (descending direction), knowing that the best value is currently in the upper side of the search region. The routine follows by evaluating the application's execution with different threads, reducing the search space by half in each iteration (lines 19-45). The algorithm, then, evaluates if there is an improvement in executing the application with med threads. If it improves, (p', e') receives the tuple used on the evaluation and v_{best} is replaced by v . Finally, the algorithm returns the best number of threads and the best value for the considered metric.

Algorithm 4 *Binary Search (BS)*

Input: $app \leftarrow$ Path to the application's binary;
 $opt \leftarrow$ Optimization metric ["Performance", "Energy", or "EDP"];
 $(p, e) \leftarrow$ Solution's tuple;
 $cores \leftarrow$ Cores' type ['P', 'E'];
 $inf \leftarrow$ Inferior limit;
 $sup \leftarrow$ Superior limit;
 $v_{best} \leftarrow$ best value for opt metric found so far.

Output: $(p', e'), v_{best}$, being the current best number of threads and its value for opt metric, respectively.

```
1:  $(p', e') \leftarrow (p, e)$ 
2:  $invert \leftarrow 0$                                 ▷ Ascending order
3: if ( $cores == 'P'$ ) then                                ▷ P-cores
4:   if ( $inf == sup$ ) then
5:     return ( $sup, e$ ),  $v_{best}$ 
6:   end if
7:   if ( $p == sup$ ) then
8:      $invert \leftarrow 1$                                 ▷ Descending order
9:   end if
10: else                                                    ▷ E-cores
11:   if ( $inf == sup$ ) then
12:     return ( $p, sup$ ),  $v_{best}$ 
13:   end if
14:   if ( $e == sup$ ) then
15:      $invert \leftarrow 1$                                 ▷ Descending order
16:   end if
17: end if
18:  $med \leftarrow sup$ 
19: while ( $sup - inf > 1$ ) do
20:    $med \leftarrow (sup + inf)/2$ 
21:   if ( $cores == 'P'$ ) then                                ▷ P-cores
22:      $v \leftarrow RunProgram(app, opt, (med, e))$ 
23:   else                                                    ▷ E-cores
24:      $v \leftarrow RunProgram(app, opt, (p, med))$ 
25:   end if
26:   if ( $v < v_{best}$ ) then
27:     if ( $cores == 'P'$ ) then
28:        $(p', e') \leftarrow (med, e)$ 
29:     else
30:        $(p', e') \leftarrow (p, med)$ 
31:     end if
32:      $v_{best} \leftarrow v$ 
33:     if ( $invert$ ) then
34:        $sup \leftarrow med$ 
35:     else
36:        $inf \leftarrow med$ 
37:     end if
38:   else
39:     if ( $invert$ ) then
40:        $inf \leftarrow med$ 
41:     else
42:        $sup \leftarrow med$ 
43:     end if
44:   end if
45: end while
46: return ( $p', e'$ ),  $v_{best}$ 
```

Time Complexity Analysis. It is evident that the time complexity of our proposal depends on the following two procedures:

- *Filter.* To determine the promising region, this procedure can execute at most $\lceil \log(n) \rceil$ times, where n is the number of kind of cores most available in the system ($n = \max(|P - cores|, |E - cores|)$). Thus, resulting in a time complexity of $\mathcal{O}(\log(n))$.

- *Binary Search.* Consider the worst case of *Filter*, in which it executed $\lceil \log(n) \rceil$ times. In this case, the size of the promising region will be $2^{\log(n)} - 2^{\log(n)-1}$ which can be simplified to $2^{\log(n)-1}$. As in each iteration, the search space is logarithmically reduced, the resulted time complexity

is $\mathcal{O}(\log(2^{\log(n)-1}))$, which can be shorten to $\mathcal{O}(\log(n))$.

Therefore, the overall time complexy of our proposal is given by $\mathcal{O}(\log(n)) + \mathcal{O}(\log(n))$, which is asymptotically equals to $\mathcal{O}(\log(n))$.

A. Implementation Details.

We implemented *HyTuning* with Python version 3.10.6. To allow for proper configuration of the number of threads and allocate threads to cores, we used the `OMP_NUM_THREADS` and `GOMP_CPU_AFFINITY` environment variables. For P-cores, we first allocate on the physical cores, and then in the logical cores. We also set `OMP_PROC_BIND=TRUE` to inhibit thread migration. To collect the execution time, we used `time.time()` function from the `time` module from Python3. Also, to gather energy results, we used the Linux Perf monitoring tool [18].

IV. METHODOLOGY

Evaluated Benchmarks. We used seven kernels from the NAS Parallel Benchmark Suite version 3.4.2 parallelized with OpenMP [19]: Block Tri-diagonal (BT), Conjugate Gradient (CG), Embarassingly Parallel (EP), Fourier Transform (FT), Integer Sort (IS), Lower-Upper (LU), and Multi-Grid (MG); as well as other four parallel algorithms Jacobi Method (JA), LULESH (LU), NBody (NB) and Poisson (PO). All of them were written in C and compiled with GCC 8.3.0 using the -O3 flag.

Execution Environment. We consider a system with the Intel Core i9-12900K Alder Lake, as described in Section II. The system is equipped with 32GB of RAM and operates on Ubuntu v. 22.04 distribution and kernel version 5.19.

Compared Solutions. We compare *HyTuning* with other four strategies: **Full**, the regular execution of a parallel application, using all the available cores; **P-Full**, all P-cores executing threads, but E-cores idle; **E-Full**, all E-cores executing threads, but P-cores idle; and **Oracle**, the best solution found during our DSE, which evaluated all possible combinations of number of threads in P-cores and E-cores.

V. EXPERIMENTAL RESULTS

A. Design Space Exploration

We performed a DSE considering all the evaluated applications executed with all combinations of the number of threads in P-core and E-core, which took a total time of 12.4 hours.

In Table I, we depict the configurations that provide the best results regarding execution time, energy, and EDP for each evaluated application. The algorithms are sorted, in ascending order, by their thread-level parallelism (TLP) degree extracted through the Intel VTune Profiler [20]. The solutions for each algorithm are formatted as (#P-cores, #E-cores), representing the number of threads in the respective types of cores. From the presented results, we can argue the following:

- *Using the maximum number of threads will not always deliver the best outcome.* Only EP benefited from using all the available resources when optimized for execution time;
- *Higher TLP does not mean the application will use all the available cores.* Due to computation unbalancing of the application's threads and the heterogeneous nature of AMPs, using the application's TLP to decide the best number of

TABLE I
BEST SOLUTIONS FOUND IN THE DSE REGARDING EXECUTION, ENERGY, AND EDP, ALONG WITH THE THREAD-LEVEL PARALLELISM (TLP) OF EACH APPLICATION.

Application	TLP	Time	Energy	EDP
EP	66.4	(16,8)	(0,8)	(16,8)
FT	74.8	(14,0)	(0,8)	(5,0)
MG	76.4	(6,0)	(0,5)	(0,5)
IS	76.5	(16,0)	(0,8)	(0,8)
BT	78.5	(16,6)	(0,8)	(0,8)
CG	80.2	(13,0)	(0,8)	(0,8)
LU	80.3	(16,5)	(0,8)	(0,8)
JA	80.4	(8,0)	(0,1)	(5,0)
PO	83.9	(16,7)	(0,8)	(15,1)
LULESH	84.1	(8,0)	(0,8)	(8,0)
NB	85.4	(4,0)	(0,5)	(3,0)

threads is not a good choice. Our experiments showed no pattern of TLP with the number of threads in P-cores and E-cores;

- *The types of cores mostly used depend on the metric being optimized.* We observed that when optimizing for execution time and energy, the applications will be better executed using mostly P-cores and E-cores, respectively. However, it does not happen for EDP since some applications benefit more by using mostly P-cores (e.g., EP and FT) while others use E-cores (e.g., MG and IS).

We mainly considered the last observation to design *HyTuning* since its search procedures take into account the specific metric being optimized. With that, we are able to cut 74% of the search space.

B. HyTuning Performance and Convergence Pattern

Let us show the *HyTuning* benefits by comparing it with the strategies mentioned in Section IV) (*Full*, *P-Full*, *E-Full*, and *Oracle*). For that, we show in Fig.4, the execution time, energy, and EDP (y-axis) for each application (each group of bars) when optimized by the compared strategies (bars with different colors) and the geometric mean (last group of bars). Results are normalized by the regular parallel execution (*Full*), so the lower, the better.

It is clear that *HyTuning* outperforms *Full*, *P-Full*, and *E-Full* strategies, being closer to the *Oracle* in all the optimized metrics. Overall, it is better in execution time, energy and EDP, respectively, on average $1.14\times$, $2.10\times$, and $1.75\times$ than *P-Full*, $1.79\times$, $1.49\times$, and $1.94\times$ than *E-Full*, and $1.27\times$, $2.66\times$, and $2.46\times$ than *Full*. Regarding the *Oracle* it is only $0.99\times$, $0.95\times$, and $0.96\times$ far, on average of the three respective metrics.

Besides *HyTuning* delivering a high-performance solution, we can also show it is efficient in finding the best solution. For that, we show, in Fig. 5, the *HyTuning*'s convergence pattern when it optimizes the BT application for the three evaluated metrics (lines with different colors). In x-axis, we show the number of evaluated solutions during the search process, and in y-axis, we show the execution time of each solution normalized to the first evaluate solution (the lower the value, better the result is). Based on such results, we observed that:

- *HyTuning needs only a few number of steps to find high-quality solutions.* It takes only four steps to find solutions that significantly improves over the initial solution in all the evaluated metrics (91% in execution time, 42% in energy, and 99% in EDP);

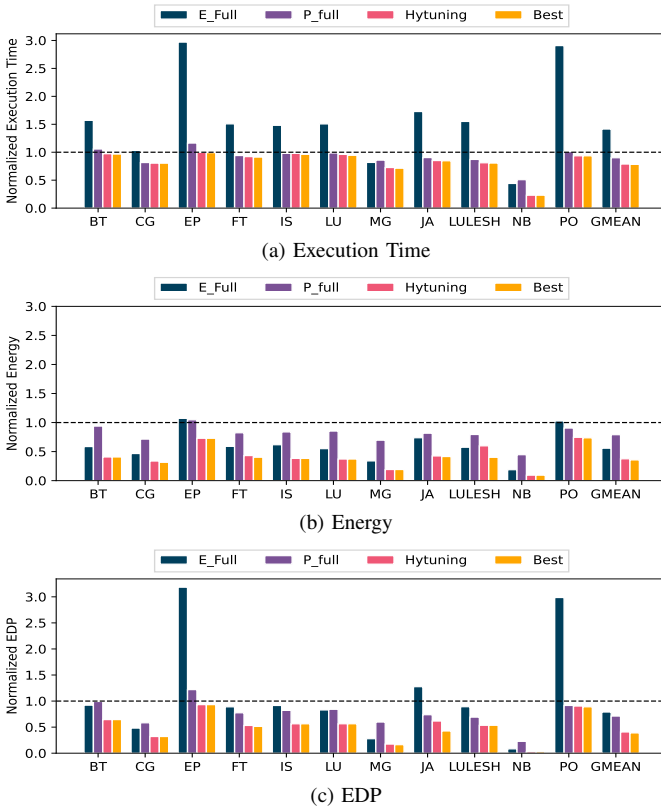


Fig. 4. *HyTuning*'s results in comparison to *P-Full*, *E-Full*, and *Oracle* solutions. Results are normalized by *Full*, so the lower, the better.

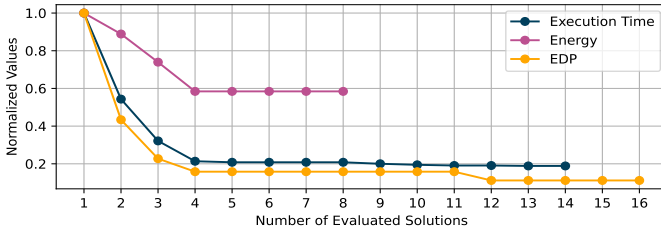


Fig. 5. *HyTuning*'s convergence when optimizing the BT application.

• *The number of evaluated solutions changes according to the optimized metric.* While it evaluated only four solutions to find the best one for energy, it evaluated 11 and 12 for execution time and EDP. Notice that, although it finds the best solution for the just mentioned number of evaluations, it follows searching to verify whether there is an even better solution. In total, *HyTuning* evaluates 8, 14, and 16 solutions for the respective metrics, which represents a reduction of 94%, 89%, and 88% in the search space.

Regarding the execution time, in the worst case of *HyTuning* search process (when optimizing BT for EDP), it takes about 1.46 minutes, and the total time spent on searching for the ideal configuration takes 10.61 minutes. Overall, considering all the evaluated applications, it takes an average optimization time of 7.01 minutes.

VI. CONCLUSION

This work proposes *HyTuning*, a heuristic algorithm to find the best number of threads to run any parallel application on AMP Alder Lake processors. We evaluated the experiments on an Intel Core i9-12900K and compared our solution to three

other strategies. Our results show that *HyTuning* outperforms the *P-Full*, *E-Full*, and *Full* strategies by at least $1.14\times$, $1.49\times$ and $1.75\times$ in execution time, energy, and EDP, respectively. For future works, we intend to evaluate *HyTuning* performance on other AMPs, such as the Apple M1, Intel Core i7 12700, and Intel Core i5 12600.

ACKNOWLEDGMENT

This study was financed in part by the CAPES - Finance Code 001, FAPESP grant #2021/06825-8, FAPERGS, and CNPq. Some experiments used the PCAD infrastructure, <http://gppd-hpc.inf.ufrgs.br>, at INF/UFRGS.

REFERENCES

- [1] J. C. Saez and M. Prieto-Matias, "Evaluation of the intel thread director technology on an alder lake processor," in *ACM SIGOPS APSYS*, 2022, pp. 61–67.
- [2] C. Bilbao, J. C. Saez, and M. Prieto-Matias, "Flexible system software scheduling for asymmetric multicore systems with pmsched: A case for intel alder lake," *CCPE*, p. e7814, 2023.
- [3] K. K. Pusukuri, R. Gupta, and L. Bhuyan, "Thread reinforcer: Dynamically determining number of threads via os level monitoring," *IISWC*, 11 2011.
- [4] D. De Sensi, M. Torquati, and M. Danelutto, "A reconfiguration algorithm for power-aware parallel applications," *ACM Trans. Archit. Code Optim.*, vol. 13, no. 4, dec 2016.
- [5] M. Bari, N. Chaimov, A. Malik, K. Huck, B. Chapman, A. Malony, and O. Sarood, "Arcs: Adaptive runtime configuration selection for power-constrained openmp applications," 09 2016, pp. 461–470.
- [6] A. F. Lorenzon, C. C. De Oliveira, J. D. Souza, and A. C. S. Beck, "Aurora: Seamless optimization of openmp applications," *TPDS*, vol. 30, no. 5, pp. 1007–1021, 2018.
- [7] H. Rocha, J. Schwarzrock, M. Pereira, L. Schnorr, P. Navaux, A. Lorenzon, and A. C. S. Beck Filho, "Attune: A heuristic based framework for parallel applications autotuning," in *SBESC*. SBC, 2020, pp. 151–156.
- [8] J. Schwarzrock, H. M. G. d. A. Rocha, A. C. S. Beck, and A. F. Lorenzon, "Effective exploration of thread throttling and thread/page mapping on numa systems," in *HPCC/SmartCity/DSS*. IEEE, 2020, pp. 239–246.
- [9] J. Schwarzrock, H. M. G. D. A. Rocha, A. F. Lorenzon, and A. C. S. Beck, "Smoothing on dynamic concurrency throttling," in *IPDPSW*. IEEE, 2022, pp. 962–971.
- [10] M. K. Moori, H. M. G. de A. Rocha, J. Schwarzrock, A. F. Lorenzon, and A. C. S. Beck, "Improving the efficiency of graph algorithm executions on high-performance computing," *CCPE*, vol. 35, no. 18, p. e7419, 2023.
- [11] A. F. Lorenzon, A. L. Sartor, M. C. Cera, and A. C. S. Beck, "Optimized use of parallel programming interfaces in multithreaded embedded architectures," in *ISVLSI*. IEEE, 2015, pp. 410–415.
- [12] J. Lee, H. Wu, M. Ravichandran, and N. Clark, "Thread tailor: Dynamically weaving threads together for efficient, adaptive parallel applications," in *ISCA*, ser. ISCA '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 270–279.
- [13] W. Wang, J. W. Davidson, and M. L. Soffa, "Predicting the memory bandwidth and optimal core allocations for multi-threaded applications on large-scale numa machines," in *HPCA*, 2016, pp. 419–431.
- [14] S. Malave and S. Shinde, "Reinforced mantaray foraging optimiser for determining the optimal number of threads in multithreaded applications," *IJISAE*, vol. 10, no. 3s, pp. 17–26, 2022.
- [15] V. Dalakoti and D. Chakraborty, "Apple m1 chip vs intel (x86)," *IJRD*, vol. 7, no. 5, pp. 207–211, 2022.
- [16] T. Yu, R. Zhong, V. Janjic, P. Petoumenos, J. Zhai, H. Leather, and J. Thomson, "Collaborative heterogeneity-aware os scheduler for asymmetric multicore processors," *IEEE TPDS*, vol. 32, no. 5, pp. 1224–1237, 2021.
- [17] A. Yue and S. Mehta, "An application-oriented approach to designing hybrid cpu architectures," in *ISPASS*, 2023, pp. 92–102.
- [18] T. Gleixner and I. Molnar, "Linux perf," <https://perf.wiki.kernel.org>, 2023.
- [19] D. Bailey et al., "The nas parallel benchmarks," *Int. J. High Perform. Comput. Appl.*, vol. 5, no. 3, p. 63–73, sep 1991.
- [20] Intel®, "Intel® vtune™ profiler user guide," <https://www.intel.com/content/dam/develop/external/us/en/documents/vtune-profiler-user-guide.pdf>, 2023.